Computer Graphics (CSED451-01)
Assignment 1: 2D Drawing

Team Baguette - *Github Repository*

BLAIS Vladimir
CSED
49005916 - vblais

PICCAND Eliott
CSED
49005903 - piccandeliot

Development environment

To develop our program, we used VSCodium (with the clangd extension) for code editing, and Visual Studio 2026 (Community) for building. To understand the source code, the reader must be familiar with modern C++ features (C++ 23).

Program design and implementation

Our program contains several features, including :

- basic player controls;
- world border collisions;
- missiles aim, fire, travel and explosion;
- camera shaking on explosions;
- ship trail;

and some features not visible by the players, but useful for development :

- entity system (inheritance based);
- event system;
- input system;

In order to compile our program, the only additional requirement is enabling C++23 features (accessible on Visual Studio under `Project > Properties > Configuration Properties > General > C++ Language Standard`). This choice was made in prevision for the following assignments, were we will probably use the C++23 `std::ranges` and `std::views` features to shorten development time. So far, the only part of our code requiring C++23 is `#include <print>` and `std::println` in `Src/Main.cpp`.

To summarize how our program works, we can use the following pseudo-code (see Algorithm 1)

Algorithm 1. – Game Main Loop

```
1:  function Main-Loop()
2:      while ¬ Window-Should-Close() do
3:          ▷ processes all events that occurred during the last frame
4:          Process-Events()
5:
6:          ▷ processes all events that occurred during the last frame
7:          Update-Entities()
8:
9:          ▷ render everything (this is a constant function, no update occurs here)
10:         Render()
11:
12:         ▷ display the rendered content on the screen
13:         Swap-Framebuffers()
14:
15:         ▷ prepare every user input that occurred during the frame for the next update call
16:         Process-Inputs()
17:     end
18: end
```

This is a usual game main loop. Sometime, games place the `Process-Events()` part at the end of the loop, but we decided to place it at the beginning, since it has no impact on the behavior of the program, and allow to defer calls during the initialization (even if we do not need that so far, we might need it in the future).

We made the world square, but the program support any rectangle shape. This can be set by changing the `WORLD_WIDTH` and `WORLD_HEIGHT` constants inside `Utils/Constants.h`. Additionally, we added a margin around the work because we thought it looks nicer. This can be removed (such matching the exact assignment requirement) by setting the `WORLD_DISPLAY_MARGIN` constant to `0.0f`.

This is how we implemented every player-visible feature :

Basic player controls

To make the ship move we divided the work in several steps - all occurring during the ship entity update method. First, the program update the ship speed state and orientation based on the user keyboard using the input system. Then, it compute a unit vector indicating in which direction the ship is moving. Then, we add to the position of the ship this vector, multiplied by the ship's speed constant and the delta time[1]. Finally, it performs collisions checks.

World border collisions

Since the only collision that can occur in the game is the one against the world border, and that this world border is a simple square, we implemented the collision by simply computing a rough rectangle hit box around the ship (based on its position, scale and rotation), then the program check if this hit box goes outside of the world border, and in that case, compute the

[1]The delta time (`deltaTime` in the code), represent the duration in seconds of the previous frame. Multiplying the speed of moving parts of the game by this make the displayed speed independent of the current framerate, which vary from one computer to another.

distance it went off the map, then subtract this distance to the ship position.

Missiles

Aim & Fire : During the Ship entity update method, the program check for mouse input using the input system. Depending on the buttons' states, the ship target position and a flag indicating whether the player is aiming are updated. Then, if the player is aiming (see Fig. 1) and the left button is released, a `FireEvent` is sent to the event system, which will spawn an new missile entity on the next frame.

Travel : Each missile move the same way the ship does but user inputs cannot update its speed nor its direction. Missiles flight in a straight line, until they are close enough[2] to their target, at which point they trigger a `TargetReachedEvent` which delete the missile entity, and spawn a new explosion entity. Finally, missiles are rendered as a simple point primitive (`GL_POINT`), so rotation and scaling does not impact them. Thus, we did not use any transformation matrices and simply create a vertex at the missile position, setting its size by changing OpenGL point size with `glPointSize(float radius)` (see Fig. 2).

Explosion : Explosion update are quite simple : its radius is increased each frame, until the maximum is reached (`EXPLOSION_MAX_RADIUS`), then an `ExplosionDoneEvent` is sent, deleting the explosion entity.

The render part is a bit more complex : the program draw 3 layers, 3 times se same mesh, but with different scale, rotation and color. Layer's scale are determined by the current explosion radius : the first layer has that radius, and then, each layer scale is halved regarding the previous one. For their rotation, they are set randomly on the entity creation. Finally, their color is each different (first layer's color being red, last yellow and middle orange), but also varies with the explosion radius : at the beginning, each layer is white, then gradually blend with its own color, simulating an initial flash (see Fig. 4).

Camera Shake

On triggering the `TargetReachedEvent`, the camera start shaking. This is done by an algorithm like Algorithm 2

Algorithm 2. – Camera Shake

```
1:  function SHAKE()
2:      ▷ Crate a vector of length INTENSITY with a random
        orientation
3:      offset ← Random-Unit-Vector() × INTENSITY
4:
5:      while length(offset) > MIN_SHAKING do
6:          ▷ Flip the offset
7:          offset ← ROTATE(offset, 180°)
8:
9:          ▷ Slightly Rotate the offset by some random angle
10:         angle ← random(−60°, 60°)
11:         offset ← ROTATE(offset, angle)
12:
13:         ▷ Decrease the offset intensity
14:         offset ← offset × INTENSITY_DECAY
15:     end
16: end
```

Ship Trail

To display the ship foam trail (see Fig. 3), we decided to store the ship position at regular interval[3], and to display a point (`GL_POINTS` primitive) on each of those positions, with a different size and opacity depending on how long the position has been stored.

End-user guide

The game starts immediately on running the executable. The player can change the ship speed with the `W` and `S` keys (respectively increasing and decreasing the boat speed), and rotate it using the `A` and `D` keys (turing the boat respectively left and right by 15°). In addition, the player can shoot missiles with his mouse : pressing the left click enable aiming mode which displays a ray toward the target. In aiming mode, 2 actions ar possible : cancel fire by clicking (press and release) the right click, or fire by releasing the left click. Fullscreen can be toggle by clicking the F11 key.

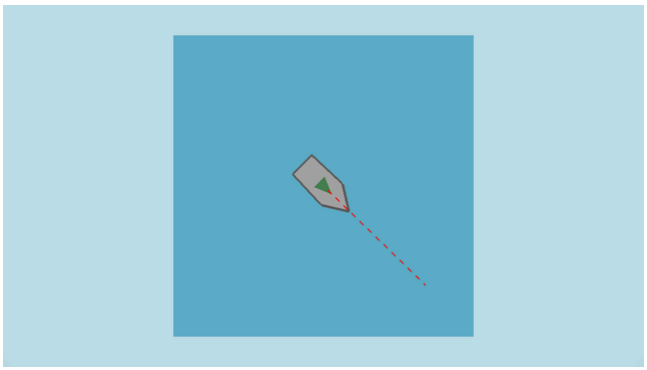

Fig. 1. – Ship aiming (target at the end of the dashed line)

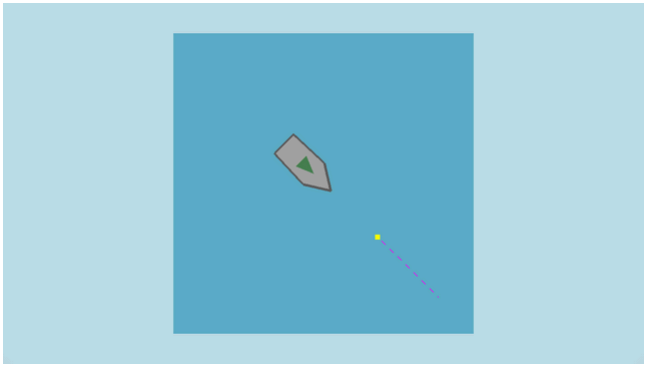

Fig. 2. – Ship firing a missile (the yellow point, target at the end of the dashed line)

[2] because position are floating point numbers and time steps are discrete trying to check if the missile reach the exact target position will always fail. Instead, the program check if the distance between the missile and the target is near 0 (configurable with the `MISSILE_TARGET_ERROR_MARGIN` constant)

[3] We implemented that using a cyclic queue data structure - since there is only a limited amount of position needed each frame - to avoid allocating memory each frame.

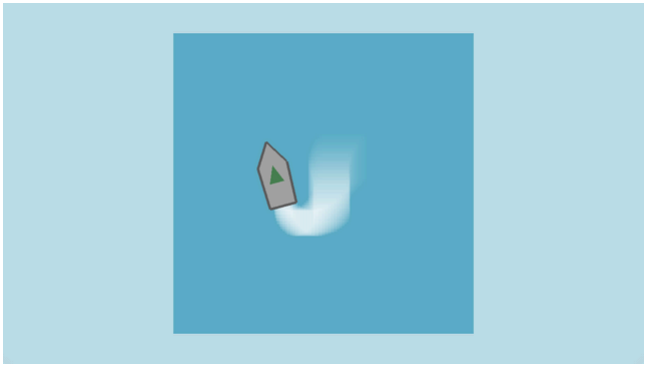

Fig. 3. – Ship with its foam trail behind

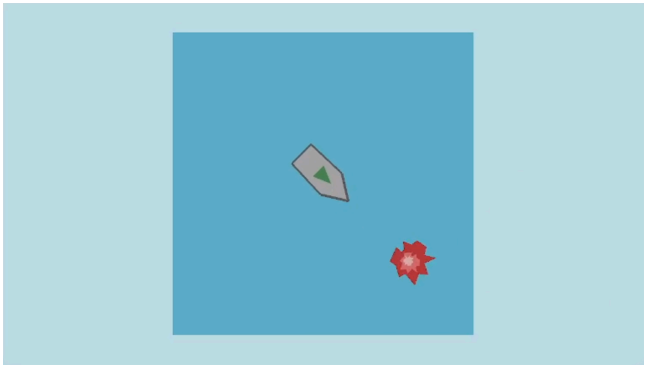

Fig. 4. – Final stage of the missile's explosion

Discussions/Conclusions

During the development, we didn't encountered much issued. However, we had to learn how to use some OpenGL functions such as `glPushMatrix()` and `glPopMatrix()` to make every model matrices properly bind to the right vertices. Moreover, we had to use AI for one part of the code since we didn't find a good tutorial explaining how to implement this feature, but this is more a C++ issue than a Graphics Computing one (see the later section about this topic)

References

Every part of the code is original, but the camera shaking part mechanic is greatly inspired by @miklatov answer on this Stack Exchange discussion.

AI-assisted coding references

The only part of this program where AI was used was to make the custom `CyclicQueue` iterable. So `CyclicQueue::Iterator`, `CyclicQueue::begin()` and `CyclicQueue::end()` were generated using an LLM.